

Day 12 - Testing and Debugging: pytest, Type Hints

Day 12 — Testing and Debugging: pytest, Type Hints

Objectives

- Understand, in concrete terms, why automated tests exist and what problem they actually solve
- Write real tests using `pytest`: plain assertions, checking that an error is correctly raised, fixtures, and parametrized tests
- Understand what type hints are, what they genuinely give you, and what they deliberately do *not* do
- Learn to use a real debugger instead of relying only on scattering `print()` statements everywhere

What you've been doing all along, and why it isn't quite enough

Every single exercise file this whole course has ended with a block of code that prints `PASS` or `FAIL` for each function you wrote. That block was, quite literally, a tiny, hand-built, extremely minimal *testing framework* — you've been practicing the core idea of automated testing since Day 1, without a name attached to it yet. Today gives that idea its proper name and its real, professional tooling.

Why do automated tests matter so much, especially at a job? Right now, your programs are small enough that you can just run them and glance at the output to see if something's obviously wrong. But imagine a program with thousands of lines, worked on by a whole team, where changing one small function somewhere might accidentally break something completely different, in some other far-away part of the program, that you didn't even think to check by hand. Automated tests are what catch that kind of accidental breakage **immediately** — the instant you make a change, before you ship it — rather than someone discovering it days or weeks later, in production, potentially affecting real users. The core mindset shift for today: **tests aren't just there to prove your code works once, right now — they exist so that any future change to the code gets checked against everything it might have broken, automatically, forever.**

Setting up pytest

```
pip install pytest
```

`pytest` is a separate, third-party tool (recall from Day 11: not part of Python's built-in standard library, so it must be installed with `pip`) that automatically finds and runs your tests. It looks for files named `test_*.py` or `*_test.py`, and inside those files, for individual functions whose names start with `test_`. You don't need any special import to write a basic test — just Python's own built-in `assert` keyword:

```
# test_math_utils.py
def add(a, b):
    return a + b

def test_add_positive_numbers():
```

```

assert add(2, 3) == 5

def test_add_negative_numbers():
    assert add(-1, -1) == -2

```

`assert some_condition` is a statement meaning "if `some_condition` is false, immediately raise an error (specifically, an `AssertionError`) right here; if it's true, do absolutely nothing and continue to the next line." A test function "passes" simply by running to the end without any `assert` inside it ever failing.

Run your tests from the terminal with:

```

pytest                                # automatically finds and runs every test_*.py file in the current folder
pytest test_math_utils.py -v          # run just one specific file, with -v ("verbose") showing each individual test
pytest -k "negative"                  # only run tests whose function name contains the word "negative"

```

When an `assert` fails, `pytest` automatically shows you a detailed, helpful breakdown of exactly which values didn't match and why — you don't need to write your own custom failure message the way some older testing tools require.

Testing that an error is correctly raised

Sometimes the entire *point* of a test is to confirm that your code correctly raises an error under bad input — for that, `pytest` provides `pytest.raises`:

```

import pytest

def divide(a, b):
    if b == 0:
        raise ValueError("cannot divide by zero")
    return a / b

def test_divide_by_zero_raises():
    with pytest.raises(ValueError):
        divide(10, 0)

```

Read this as: "run the code inside this `with` block, and specifically expect a `ValueError` to be raised somewhere in there — if it is, this test passes; if the code runs to the end *without* raising that error, the test fails, because the whole point was confirming that the error genuinely happens."

Fixtures — reusable setup code, matched to your tests by name

A **fixture** is a small, separate function that prepares some piece of data or setup you want available to one or more of your tests, without repeating that setup code inside every single test function:

```

import pytest

@pytest.fixture
def sample_account():
    return {"owner": "Ana", "balance": 100}

def test_deposit(sample_account):
    sample_account["balance"] += 50
    assert sample_account["balance"] == 150

```

The mechanism here, precisely: `pytest` notices that `test_deposit` has a parameter named `sample_account`, sees that there's a fixture function *also* named `sample_account`, and automatically calls that fixture and hands you its return value as the argument — this isn't hidden magic, it's just `pytest` matching parameter names to fixture names for you.

Parametrize — running one test body against many different inputs

```
import pytest

@pytest.mark.parametrize("a, b, expected", [
    (2, 3, 5),
    (-1, 1, 0),
    (0, 0, 0),
])
def test_add(a, b, expected):
    assert add(a, b) == expected
```

This one function definition actually runs as **three separate, individually-reported tests** — one for each tuple of values in that list — which is far better than manually writing three separate `assert` lines inside one test, because if, say, only the second combination of inputs fails, `pytest`'s output tells you exactly which one failed, rather than you having to guess which `assert` line among several was the culprit.

What actually makes a test a *good* test

- **Test one specific behavior per test function** — a test named `test_withdraw_insufficient_funds` should check exactly that one scenario, not sneak in an unrelated check about deposits at the same time.
- ****Test what a function *does*, not the specific way it happens to be written internally**** — check its return value or its externally-visible effect, not private implementation details that might reasonably change later without the actual behavior changing.
- **Cover the edge cases, not just the "happy path"** — this entire course's exercises have been quietly training this habit already: empty lists, missing dictionary keys, zero, negative numbers. Keep doing this deliberately now that you know it has a name.
- **Tests should give the same result every single time they run** — never depend on the exact current time, on genuinely random values (unless you deliberately fix, or "seed," the randomness), on an actual network connection, or on tests happening to run in one particular order.

Type hints — documentation that a tool can actually double-check for you

```
def add(a: int, b: int) -> int:
    return a + b

def greet(name: str, times: int = 1) -> list[str]:
    return [f"Hello, {name}!" for _ in range(times)]
```

The `: int` after a parameter name, and the `-> int` after the closing parenthesis, are called **type hints** (or "type annotations") — they state what type each parameter is expected to be, and what type the function is expected to return.

Crucially, type hints do absolutely nothing at all while your program is actually running — recall from Day 1 that Python is dynamically typed, and that hasn't changed; Python will happily run `add("a", "b")` without a single complaint, type hints notwithstanding, and simply return `"ab"` (since `+` on two strings means "join them together"). So what are type hints actually *for*, if Python itself ignores them? Three real, practical things: first, they're documentation that lives directly in the code itself, right where you're reading it, instead of drifting out of sync in some separate document nobody remembers to update. Second, your code editor can read them and warn you, as you type, if you're about to pass the wrong kind of value somewhere. Third — and this is the big one, professionally — a completely separate tool called a **type checker** (`mypy` and `pyright` are the two most common ones) can scan your entire codebase *before you ever run it* and catch a whole category of mistakes ahead of time, which is exactly how the Python ecosystem compensates for not having a true compiler that would otherwise catch these errors automatically, the way some other languages do.

A shape you'll see constantly, worth knowing today:

```
from typing import Optional    # or, in modern Python (3.10+), just write `X | None` directly, no imp

def find_user(user_id: int) -> Optional[dict]:    # equivalently: -> dict | None
    ...
```

`Optional[X]` (or `X | None`) means "this returns a value of type X, OR it might return `None` instead" — this is the exact same "might not find anything" situation from Day 3's `dict.get()`, now stated explicitly and visibly right in the function's signature, so that anyone calling this function is warned, right there, that they need to handle the possibility of getting `None` back.

At most real jobs using Python today, you'll be expected to add type hints to new code you write — it's considered a standard, expected practice in most modern, professional Python codebases now, not an optional extra.

Debugging with a real debugger, instead of only `print()`

Scattering `print(some_variable)` statements throughout your code to see what's happening works, and you've likely already done exactly this while working through earlier days — but it has a real limitation: you have to *guess in advance* what you'll need to see, add print statements for exactly that, run the program, and if you guessed wrong or need to see something else too, edit and re-run all over again. Python's built-in debugger lets you pause your program mid-execution and look at *anything*, interactively, without needing to have predicted it beforehand:

```
def buggy_function(items):
    breakpoint()          # execution PAUSES here and drops you into an interactive prompt
    total = sum(items)
    return total / len(items)
```

When execution hits `breakpoint()`, you'll get an interactive (`Pdb`) prompt right there in your terminal. A few commands worth knowing today: `n` (run the next line), `s` (step *into* a function call, rather than over it), `c` (continue running normally until the next breakpoint, or the end), `p some_variable` (print the current value of any variable you're curious about), `l` (show the surrounding lines of code for context), `q` (quit the debugger entirely). This gives you the power to inspect *any* variable, at that exact moment, and even test out a

potential fix interactively before committing to writing it into your actual code.

Most code editors (VS Code, PyCharm) also provide a visual debugger — you click directly in the margin next to a line of code to set a breakpoint there, then run your program in a special "debug mode," and get this exact same inspection power through a graphical interface instead of a text prompt. Building the habit of reaching for a debugger, instead of defaulting to `print()`, the moment something confusing happens, is one of the single highest-value habits you can build starting this week.

Exercises

Open `calculator.py` and add type hints to every function first (see the instructions at the top of `test_calculator.py` for exactly what's expected). Then open `test_calculator.py` and fill in each `# TODO` test. Run everything with `pytest test_calculator.py -v` from this folder and read the output.